

Diseño e Implementación de la Arquitectura de Software



Nombre del Proyecto: **Explora mas**

Tipo de Sistema: **Aplicación Móvil Android Nativo**

Desarrollador: **Francisco Bascur Arévalo**

Fecha: **01/04/2026**

NDICE

1. Introducción
2. Descripción del Proyecto
3. Contexto del Problema
4. Objetivos del Sistema
 - 4.1 Objetivo General
 - 4.2 Objetivos Específicos
5. Alcance del Sistema
6. Desarrollo del Sistema
7. Tecnologías Utilizadas
8. Herramientas de Desarrollo
9. Arquitectura del Sistema
 - 9.1 Arquitectura Cliente-Servidor
 - 9.2 Arquitectura Monolítica Modular
 - 9.3 Arquitectura MVVM
 - 9.4 Arquitectura en Capas
10. Base de Datos
11. API REST
12. Seguridad
13. Pruebas del Sistema
14. Beneficios del Sistema
15. Conclusión

1. INTRODUCCIÓN

El presente documento describe el diseño e implementación de la arquitectura de software del proyecto Explora Más, una aplicación móvil Android enfocada en centralizar información relacionada con actividades recreativas, culturales y urbanas.

El documento presenta las problemáticas identificadas, objetivos del sistema, tecnologías utilizadas, arquitectura seleccionada y estructura general del desarrollo del proyecto. Además, se detallan las decisiones técnicas implementadas para garantizar una aplicación organizada, mantenible y escalable.

2. DESCRIPCIÓN DEL PROYECTO

Explora Más es una aplicación móvil Android desarrollada con el objetivo de centralizar información relacionada con eventos culturales, actividades recreativas, panoramas urbanos y lugares turísticos dentro de una única plataforma digital.

La aplicación busca facilitar el acceso a información organizada y actualizada, permitiendo que los usuarios puedan descubrir actividades de interés mediante una interfaz moderna, intuitiva y amigable.

El sistema será desarrollado utilizando una arquitectura cliente-servidor basada en un backend monolítico modular y una aplicación móvil nativa desarrollada en Kotlin utilizando Jetpack Compose.

3. CONTEXTO DEL PROBLEMA

Actualmente, gran parte de la información relacionada con eventos culturales, actividades recreativas y panoramas urbanos se encuentra distribuida en múltiples plataformas, dificultando el acceso rápido y centralizado para los usuarios.

Esto genera diversos problemas:

- Dificultad para encontrar información centralizada.

- Uso de múltiples aplicaciones diferentes.
- Información desactualizada o poco organizada.
- Baja personalización de contenidos.
- Escasa integración entre actividades culturales y urbanas.

Debido a esta problemática surge la necesidad de desarrollar una plataforma móvil que permita integrar distintos tipos de actividades recreativas y culturales dentro de una única aplicación moderna y organizada.

4. OBJETIVOS DEL SISTEMA

4.1 Objetivo General

Desarrollar una aplicación móvil Android que centralice información relacionada con actividades recreativas, culturales, urbanas y turísticas mediante una plataforma moderna, organizada y escalable, permitiendo mejorar el acceso y difusión de información para los usuarios.

4.2 Objetivos Específicos

- Visualizar actividades recreativas de forma organizada y amigable.
- Gestionar eventos culturales y urbanos.
- Organizar la información mediante categorías y subcategorías.
- Implementar autenticación para usuarios y administradores.
- Centralizar contenidos provenientes de distintas fuentes.
- Aplicar arquitectura cliente-servidor mediante APIs REST.
- Implementar una arquitectura modular mantenible y escalable.
- Mejorar la experiencia de usuario mediante una interfaz intuitiva.

5. ALCANCE DEL SISTEMA

El sistema incluirá:

- Registro e inicio de sesión.
- Sistema de autenticación.
- Visualización de eventos y actividades.
- Organización por categorías y subcategorías.
- Exploración de lugares turísticos y culturales.

- Sistema de favoritos.
- Panel administrativo básico.
- Consumo de APIs REST.
- Gestión de eventos.
- Gestión de usuarios.

El sistema NO incluirá inicialmente:

- Pagos online.
- Chat en tiempo real.
- Sistema de reservas.
- Inteligencia artificial avanzada.
- Geolocalización avanzada en tiempo real.

6. DESARROLLO DEL SISTEMA

El desarrollo del sistema se realizará utilizando una arquitectura cliente-servidor compuesta por un frontend Android nativo desarrollado en Kotlin y un backend centralizado implementado con Spring Boot.

La aplicación móvil consumirá APIs REST utilizando Retrofit para comunicarse con el backend y acceder a la información almacenada en PostgreSQL.

El frontend será desarrollado utilizando arquitectura MVVM, mientras que el backend utilizará arquitectura monolítica modular y arquitectura separada en capas.

7. TECNOLOGÍAS UTILIZADAS

Frontend

- Kotlin
- Jetpack Compose
- Retrofit
- MVVM
- Corrutinas Kotlin

Backend

- Java
- Spring Boot
- Spring Data JPA
- Hibernate
- APIs REST

Base de Datos

- PostgreSQL

8. HERRAMIENTAS DE DESARROLLO

- Android Studio
- IntelliJ IDEA
- Postman
- Git
- GitHub

9. ARQUITECTURA DEL SISTEMA

9.1 Arquitectura Cliente-Servidor

La arquitectura Cliente-Servidor divide el sistema en dos componentes principales:

- Cliente (Frontend Android)
- Servidor (Backend Spring Boot)

El cliente se encarga de la interacción con el usuario, mientras que el servidor procesa las solicitudes, administra la lógica del negocio y gestiona la base de datos.

Flujo General

Aplicación Android → API REST → Spring Boot → PostgreSQL

Funciones del Cliente

- Mostrar información.
- Capturar acciones del usuario.
- Consumir APIs REST.
- Mostrar resultados al usuario.

Funciones del Servidor

- Procesar solicitudes HTTP.
- Aplicar lógica de negocio.
- Gestionar autenticación.
- Administrar acceso a datos.
- Responder información en formato JSON.

Dependencias por Módulo

Módulo	Dependencias clave
auth	Spring Security + JWT
users, events, places, categories, favorites	Web + Data JPA + Validation
Base de datos	PostgreSQL Driver
Código limpio	Lombok
Testing	Spring Boot Test

9.2 Arquitectura Monolítica Modular

El backend será desarrollado utilizando una arquitectura monolítica modular, permitiendo organizar el sistema en módulos independientes según sus funcionalidades.

Estructura Modular Propuesta

```
/backend
├── users
├── events
├── categories
├── favorites
├── places
└── auth
```

9.3 Arquitectura MVVM

El frontend Android utilizará la arquitectura MVVM (Model - View - ViewModel).

Componentes

View

Responsable de:

- Mostrar información.
- Capturar acciones del usuario.
- Observar cambios del ViewModel.

ViewModel

Responsable de:

- Gestionar estados de la interfaz.
- Solicitar datos al Repository.
- Mantener lógica de presentación.

Model

Responsable de:

- Gestionar datos.
- Consumir APIs REST.
- Representar entidades del sistema.

Flujo MVVM

View → ViewModel → Repository → API REST

Beneficios

- Separación clara de responsabilidades.
- Código más organizado.
- Mejor mantenibilidad.
- Facilita testing.

9.4 Arquitectura en Capas

El backend utilizará arquitectura separada en capas para mantener responsabilidades organizadas.

Capas del Sistema

Capa de Presentación (Controllers)

Responsable de:

- Recibir solicitudes HTTP.
- Exponer APIs REST.
- Retornar respuestas JSON.

Capa de Servicio (Services)

Responsable de:

- Aplicar lógica de negocio.
- Validar información.
- Procesar reglas del sistema.

Capa de Acceso a Datos (Repositories)

Responsable de:

- Acceso a PostgreSQL.
- Persistencia de información.
- Consultas mediante JPA.

Capa de Modelo (Entidades)

Representa:

- Usuarios.
- Eventos.
- Categorías.
- Favoritos.
- Lugares.

Relación Entre Capas

Controllers → Services → Repositories → PostgreSQL

10. BASE DE DATOS

PostgreSQL

Se seleccionó PostgreSQL debido a:

- Estabilidad.
- Compatibilidad con Spring Boot.
- Robustez en aplicaciones empresariales.

Modelo de Datos Propuesto

Usuario

- id
- nombre
- correo
- contraseña
- rol

Evento

- id
- título
- descripción

- fecha
- ubicación
- categoría

Categoría

- id
- nombre
- descripción

Favorito

- id
- usuario_id
- evento_id

Lugar

- id
- nombre
- dirección
- descripción
- coordenadas

11. API REST

El backend expondrá APIs REST para permitir la comunicación entre la aplicación móvil y el servidor.

Principales Endpoints

Autenticación

POST /auth/login

POST /auth/register

Usuarios

GET /users

GET /users/{id}

Eventos

GET /events
POST /events
PUT /events/{id}
DELETE /events/{id}

Categorías

GET /categories

Favoritos

POST /favorites
GET /favorites

Las respuestas serán entregadas en formato JSON.

12. SEGURIDAD

El sistema implementará mecanismos básicos de seguridad para proteger la información y controlar el acceso.

Medidas de Seguridad

- Autenticación mediante login.
- Control de acceso por roles.
- Validación de peticiones.
- Validación de datos en frontend y backend.
- Protección de rutas administrativas.
- Manejo seguro de credenciales.

13. PRUEBAS DEL SISTEMA

13.1 Pruebas Unitarias

Permitirán verificar el funcionamiento correcto de funciones y componentes individuales del sistema.

13.2 Pruebas API con Postman

Se utilizará Postman para validar:

- Endpoints REST.
- Respuestas JSON.
- Estados HTTP.
- Integración frontend-backend.

13.3 Validaciones Frontend

Se realizarán validaciones en la interfaz para:

- Verificar datos ingresados.
- Mejorar experiencia de usuario.
- Prevenir errores de entrada.

14. BENEFICIOS DEL SISTEMA

- Centralización de información.
- Mejor acceso a actividades culturales.
- Plataforma moderna y organizada.
- Arquitectura mantenible.
- Escalabilidad progresiva.
- Mejor experiencia de usuario.
- Separación clara de responsabilidades.
- Facilidad de mantenimiento.
- Integración sencilla con futuras funcionalidades.

15. CONCLUSIÓN

La arquitectura seleccionada para Explora Más permite desarrollar una aplicación moderna, organizada y preparada para crecer progresivamente a medida que aumenten las funcionalidades y necesidades del sistema.

La combinación de arquitectura Cliente-Servidor, arquitectura Monolítica Modular, arquitectura MVVM y arquitectura en Capas permite mantener una estructura equilibrada entre simplicidad de desarrollo, organización interna y escalabilidad futura.

En definitiva, esta solución proporciona una base sólida para el desarrollo de una plataforma móvil enfocada en centralizar información cultural, recreativa y urbana, manteniendo buenas prácticas de desarrollo de software y una arquitectura profesional desde las primeras etapas del proyecto.